



Blazor Essencial

Blazor – Manipulação de Eventos



@ Manipulação de Eventos

O Blazor suporta o modelo de programação de eventos através do **data binding** (*vinculação de dados*), que permite associar **eventos de elementos HTML**, como **@onclick** ou **@onchange**, diretamente a métodos C#, criando uma **ponte** entre a interação do usuário na **interface** e a lógica da **aplicação**, de forma reativa e eficiente.

Os componentes Blazor podem tratar eventos usando a diretiva :

```
@on{evento_DOM}="NomeDoMetodoCSharp"
```

Os eventos só funcionam em **componentes interativos**.

Exemplo de eventos mais usados: **onclick, onchange, oninput, onkeypress e onfocus**



Tratamento de Eventos com métodos nomeados

EventosDemo.razor

```
@page "/eventos-demo"
<h3>Demonstração de Eventos</h3>

<h4>Evento com Método Nomeado</h4>
<button class="btn btn-primary" @onclick="HandleClick">
    Clique Aqui
</button>

<p>Mensagem: <strong>@mensagem</strong></p>
```

```
@code {
    private string mensagem = "Aguardando um clique...";

    private void HandleClick()
    {
        mensagem = $"O botão foi clicado às {DateTime.Now:HH:mm:ss}!";
        Console.WriteLine("O método HandleClick foi executado.");
    }
}
```

O Blazor, nos bastidores, usa a API de eventos do **DOM** para capturar o evento e passá-lo para o seu código C#.

O método **HandleClick** é o manipulador de eventos que é executado quando o evento ocorre.

```
// Adicione este código ao EventosDemo.razor
<hr />
<h4>2. Evento com Expressão Lambda</h4>
<button class="btn btn-success" @onclick="() => mensagemLambda = 'Botão verde clicado!'">
    Clique (Lambda)
</button>
<p>Mensagem Lambda: <strong>@mensagemLambda</strong></p>

@code {
    // ... código anterior
    private string mensagemLambda = "Aguardando...";
}
```

- Definimos uma função anônima que é executada no clique.
- Se a lógica tiver mais de uma linha, é melhor movê-la para um método nomeado

@ Acessando Informações do Evento com EventArgs

Podemos obter detalhes sobre um evento usando um **argumento de evento**.

A classe **EventArgs** é a classe base de onde todos os argumentos de eventos se originam.

O tipo do argumento depende do evento :

- Para eventos de mouse, é **MouseEventArgs**;
- para eventos de teclado, **KeyboardEventArgs**;
- para eventos de mudança, **ChangeEventArgs**, e assim por diante...



EventArgs : Obtendo as coordenadas do Mouse

```
// Adicione este código ao EventosDemo.razor
<hr />
<h4>3. Acessando MouseEventArgs</h4>

<div class="alert alert-info" @onclick="HandleDivClick">
    Clique em qualquer lugar dentro desta área azul.
</div>

<p>@mensagemCoordenadas</p>

@code {
    // ... código anterior
    private string mensagemCoordenadas = "Clique na área acima.";

    private void HandleDivClick(MouseEventArgs e)
    {
        mensagemCoordenadas =
            $"Você clicou nas coordenadas: X = {e.ClientX}, Y = {e.ClientY}";
    }
}
```

O método aceita um parâmetro do tipo **MouseEventArgs**, e o Blazor o fornece quando o evento **onclick** é disparado.

E temos acesso a todas as propriedades do evento, como **ClientX**, **ClientY**, e também se teclas como **CtrlKey** ou **ShiftKey** estavam pressionadas



Comunicação entre Componentes com EventCallback

Podemos utilizar os recursos dos delegates **EventCallback** e **EventCallback<T>** para executar métodos de um componente **pai** a partir de um componente **filho**.

Isso é útil quando você possui **componentes aninhados** e precisar que um componente **filho** acione um método de um componente **pai** a partir de um determinado **evento**

Usar um **EventCallback** para disparar **eventos do componente filho** para o componente **pai** é uma das principais estratégias para manter o fluxo de dados e a arquitetura da sua aplicação organizados.



Comunicação entre Componentes com EventCallback

BotaoFilho.razor

```
<button class="btn btn-warning" @onclick="NotificarPai">
```

Botão no Componente Filho

```
</button>
```

```
@code {
```

```
    [Parameter]
```

```
    public EventCallback OnClicouNoFilho { get; set; }
```

```
    private Task NotificarPai()
```

```
    {
```

```
        // Invoca o callback fornecido pelo componente pai
```

```
        return OnClicouNoFilho.InvokeAsync();
```

```
    }
```

```
}
```

```
// Adicione este código ao EventosDemo.razor
```

```
<hr />
```

```
<h4>4. EventCallback - Comunicação Filho -> Pai</h4>
```

```
<BotaoFilho OnClicouNoFilho="HandleCliqueDoFilho" />
```

```
<p>@mensagemDoFilho</p>
```

```
@code {
```

```
    // ... código anterior
```

```
    private string mensagemDoFilho = "Aguardando evento do  
filho...";
```

```
    private void HandleCliqueDoFilho()
```

```
    {
```

```
        mensagemDoFilho = "Recebi o evento do componente  
filho!";
```

```
    }
```

```
}
```


@ Modificadores de Evento e Controle de Comportamento

O Blazor oferece modificadores de evento usando a sintaxe

`@on{EVENTO}:preventDefault` e `@on{EVENTO}:stopPropagation`

1. `@onclick:preventDefault`:

Impede a ação padrão do navegador para aquele evento.

Exemplo: Clicar em um link (`<a>`) normalmente navega para uma nova página. Usar `preventDefault` impede a navegação, permitindo que apenas nosso código C# seja executado.

Clicar em um botão de submit em um formulário recarrega a página; `preventDefault` impede isso.

2. `@onclick:stopPropagation`:

Impede que o evento "`borbulhe`" para elementos pai.

Por padrão, se você tem um botão dentro de uma div e ambos têm um evento `onclick`, clicar no botão disparará o evento do botão e depois o evento da `div`, `stopPropagation` no botão impede que o evento da div seja acionado.



Exemplo:

// Adicione este código ao EventosDemo.razor

```
<hr />
```

```
<h4> Modificadores de Evento</h4>
```

```
<div class="alert alert-success" @onclick="()" => logEventos.Add('Evento da DIV externa disparado.')">
```

DIV Externa

```
<div class="alert alert-warning p-3" @onclick="()" => logEventos.Add('Evento da DIV interna disparado.')">
```

DIV Interna

```
<button class="btn btn-danger" @onclick="HandleBotaoComStop" @onclick:stopPropagation>
```

Clique (com **StopPropagation**)

```
</button>
```

```
<button class="btn btn-secondary" @onclick="()" => logEventos.Add('Evento do botão normal disparado.')">
```

Clique (Normal)

```
</button>
```

```
</div>
```

```
</div>
```

```
<ul class="mt-2">
```

```
@foreach (var log in logEventos)
```

```
{
```

```
<li>@log</li>
```

```
}
```

```
</ul>
```

```
...
```

```
@code {
```

```
// ... código anterior
```

```
private List<string> logEventos = new();
```

```
private void HandleBotaoComStop()
```

```
{
```

```
    logEventos.Add("Evento do botão com STOP disparado.");
```

```
}
```

```
}
```

The background is a gradient from dark purple at the top to deep blue at the bottom, speckled with white dots resembling a starry sky. Overlaid on this are several faint, light-colored geometric patterns. On the left, a large circular scale with degree markings from 140 to 260 is visible. To its right and further up, there are several concentric circles and arcs, some with arrows indicating a clockwise direction. These elements suggest a theme of astronomy, navigation, or scientific measurement.

FIM